

Supplementary Material for “SACK: A Stage-Adaptive Case-Knowledge System for Tabular Data Science Automation”

Xu Su

School of Computer Science and Technology, Xidian University
Xi’an, China
sxswa@stu.xidian.edu.cn

Supplementary Material Overview

This supplementary material provides additional details that are omitted from the main paper due to the page limit. Appendix A reports the benchmark inventory, task abbreviations, leakage-control protocol, and full task-level results. Appendix B describes case knowledge construction, including the case graph schema, EDInsight schema, and CoreInsight grounding. Appendix C gives the retrieval procedure, similarity signals, gating rules, and online knowledge-use statistics. Appendix D describes role-aware evidence exposure. Appendix E reports implementation and runtime details. Appendix F provides additional task-level ablation patterns and representative phase-level trace evidence.

A Benchmark and Evaluation Details

A.1 Benchmark Inventory and Task Abbreviations

This appendix expands the benchmark description in the main paper. Table S1 lists the full benchmark inventory, including task abbreviations used in task-level figures.

A.2 Historical Case Base and Leakage Control

SACK uses a historical case base constructed from past tabular or tabular-oriented data science competitions. The case base is used only as the source of historical experience for retrieval and is separated from the evaluation benchmark. All benchmark competitions used in the evaluation of the main paper are excluded from case-base construction. In online evaluation, retrieval further follows a leave-one-competition-out protocol, so that no artifact originating from the same competition can be retrieved for the evaluated task.

The current case base contains 69 historical competitions. These competitions cover binary classification, multiclass classification, regression, forecasting, matching, ranking, and domain-specific tabular prediction tasks. Historical solution pipelines from these competitions are processed offline into task profiles, structural data profiles, EDInsight profiles, phase-aware statement graphs, and grounded CoreInsights.

Table S2 summarizes the case base. The case base contains 69 historical competitions, from which we process 669 solution pipelines and extract 8,568 grounded CoreInsights. These counts are fixed before online evaluation and remain unchanged across all benchmark tasks, ensuring that retrieval is performed from a consistent historical knowledge snapshot.

Table S3 summarizes the coverage of the historical case base at the category level. The table shows the diversity of historical experience available to SACK; it does not imply that every category contributes equally to every benchmark task.

A.3 Execution Protocol and Failure Handling

All compared frameworks are evaluated under the same benchmark, runtime environment, model backend, and execution protocol. For each method–task pair, the framework starts from the original competition description and data files, constructs an end-to-end data science workflow, and generates official submissions. The evaluation source is the official Kaggle hidden-test feedback and the corresponding leaderboard ranks, rather than local validation scores.

Each method–task pair is required to obtain $K = 5$ valid submissions. A method is allowed at most 15 execution attempts on each task. An attempt is treated as valid only if it produces a submission file accepted by the official evaluation system and receives an official score. Execution failures, invalid file formats, timeout failures, and submissions rejected by the evaluation system are counted as attempts but do not receive artificial NOS or LP values.

For result quality, we report Mean-of-5 and Best-of-5 statistics over the valid official submissions obtained by each method on each task. Mean-of-5 characterizes typical repeated-run quality, while Best-of-5 characterizes the best solution trajectory reached within the retry budget. For execution reliability, we report VES@5, which measures how efficiently a method obtains the required valid submissions.

Let $r_t^{(K)}$ denote the number of execution attempts required to obtain the K -th valid submission on task t . VES@5 is defined as:

$$\text{VES}_t = \begin{cases} K/r_t^{(K)}, & \text{if the } K\text{-th valid submission is reached,} \\ 0, & \text{otherwise.} \end{cases} \quad (\text{S1})$$

This definition separates official result quality from execution reliability. Invalid attempts affect VES@5, but they are not assigned artificial NOS or LP values in task-level result reporting. When a framework fails to obtain the required valid submissions within the retry budget, the task-level Mean-of-5 and Best-of-5 entries are marked as Fail or left undefined, while VES@5 is set to 0. For aggregate failure-penalized comparisons, Fail entries are assigned 0 before macro-averaging so that inability to complete a task is reflected in the benchmark-level metrics.

A.4 Full Task-level Results

Table S4 reports the full task-level leaderboard-side metrics and execution reliability results. The table complements the aggregate benchmark-level results in the main paper and supports the task-level gain and failure-pattern analysis. For compactness, we report the official leaderboard-percentile metrics and VES@5 here; task-level NOS differences are visualized in the gain-profile figure of the main paper.

Table S1: Full benchmark inventory used in the evaluation. The abbreviation column defines the task labels used in task-level figures and tables.

Abbr.	Task	Category	Metric	Train	Test
FSS	Forecasting Sticker Sales	Time Series Analysis	MAPE	230K	98K
BPC	Backpack Prediction Challenge	Regression	RMSE	300K	200K
BPRD	Binary Prediction with a Rainfall Dataset	Classification	AUC	2,190	730
PPLT	Predict Podcast Listening Time	Regression	RMSE	750K	250K
PCE	Predict Calorie Expenditure	Regression	RMSLE	750K	250K
POF	Predicting Optimal Fertilizers	Classification	MAP@3	750K	250K
PIE	Predict the Introverts from the Extroverts	Classification	Accuracy	18K	6,175
BCBD	Binary Classification with a Bank Dataset	Classification	AUC	750K	250K
PBMS	Predicting the Beats-per-Minute of Songs	Regression	RMSE	524K	174K
PRAR	Predicting Road Accident Risk	Regression	RMSE	517K	172K
GGGB	Ghouls Goblins and Ghosts Boo	Classification	Accuracy	371	529
TBOP	Tmdb Box Office Prediction	Regression	RMSLE	3,000	4,398
NYCTFP	New York City Taxi Fare Prediction	Regression	RMSE	55M	10K
BSD	Bike Sharing Demand	Time Series Analysis	RMSLE	10.9K	6,493
CD	Cat in the Hat	Classification	AUC	300K	200K
VPP	Ventilator Pressure Prediction	Regression	MAE	6.04M	4.02M
CMIDSS	Child Mind Institute Detect Sleep States	Time Series Analysis	AP	14K	N/A
PSSDP	Porto Seguro Safe Driver Prediction	Classification	Normalized Gini	595K	893K
SCTP	Santander Customer Transaction Prediction	Classification	AUC	200K	200K
ST	Spaceship Titanic	Classification	Accuracy	8,693	4,277

Table S2: Statistics of the historical case base used by SACK.

Statistic	Value
Historical competitions	69
Historical solution pipelines	669
Extracted CoreInsights	8568
Average pipelines per competition	9.67
Average CoreInsights per pipeline	12.85

B Case Knowledge Construction Details

This appendix provides additional details on how SACK constructs historical cases as reusable case knowledge. It clarifies the design choices behind the case representation in the main paper: how data-side analytical signals are organized as EDAInsight, how solution-side reusable units are extracted as grounded CoreInsights, and how these objects are materialized into a unified case-centric knowledge graph.

B.1 Case Knowledge Graph Schema

The graph is case-centric: these objects are not stored independently. Instead, they are connected through cross-layer relations that support both multi-granular retrieval and traceable reuse: competition-level nodes connect to task-profile attributes, tables and columns, and EDAInsight objects; pipeline nodes connect to statement nodes; and CoreInsight nodes connect back to their implementing statements. As a result, the graph can be traversed from high-level task semantics to implementation evidence, or from reused code evidence back to the historical insight and case from which it originates.

Figure S1 illustrates the case-centric representation used by SACK. Each historical case links competition-level task context, structural data profiles, EDAInsight profiles, phase-aware pipeline statements, library usages, grounded CoreInsights, and implementation evidence. The figure complements Table S5: the figure shows how the objects are connected, while the table summarizes the role of each object type in retrieval and evidence exposure.

Table S5 summarizes the main object types that play a direct role in SACK retrieval and evidence exposure. The implementation uses a richer RDF-based vocabulary and additional metadata fields, but these object types are sufficient to explain how the graph serves as the structural substrate of SACK.

Graph materialization is not merely a storage choice. It makes the three components of SACK compatible with one another: case construction organizes historical cases into reusable objects, stage-adaptive retrieval traverses these objects across task, data, and insight layers, and role-aware evidence exposure resolves selected historical references back to grounded implementation evidence.

B.2 EDAInsight Schema

In SACK, EDAInsight represents data-side analytical signals in a structured form for phase-aware retrieval. Rather than storing exploratory analysis as free-form text, SACK organizes these signals into two levels:

$$E_i = \{E_i^{\text{pre}}, E_i^{\text{deep}}\}, \quad (\text{S2})$$

where E_i^{pre} denotes the *preliminary EDAInsight* extracted before cleaning, and E_i^{deep} denotes the *in-depth EDAInsight* extracted after cleaning.

This distinction reflects the phase structure of tabular data science workflows. Preliminary EDAInsight captures signals that are

Table S3: Category-level coverage of the historical case base.

Category	Representative Competitions	Typical Metrics
Playground tabular series	Playground Series S3E1–S3E26, S4E1–S4E12	AUC, accuracy, RMSE, RMSLE
Tabular playground series	Tabular Playground Series-*	AUC, RMSE, RMSLE, log loss
Binary classification	Titanic - Machine Learning from Disaster, Icr Identify Age Related Conditions	Accuracy, log loss
Regression and ranking	Scrabble Player Rating, Novozymes Enzyme Stability Prediction	RMSE, MAE, correlation-style metrics
Forecasting and temporal tabular	M5 Forecasting - Accuracy, Godaddy Microbusiness Density Forecasting, Optiver Trading At the Close	RMSSE, SMAPE, RMSE
Domain-specific tabular tasks	Amp-Parkinsons-*, Lish Moa, Foursquare Location Matching	task-specific metrics
Sports and event prediction	March Machine Learning Mania-*, Nfl Big Data Bowl 2023, Big Data Derby 2022	log loss, ranking, regression metrics

Table S4: Task-level comparison across benchmark competitions on leaderboard-side metrics and execution reliability. For compactness, the table reports Mean-of-5 LP, Best-of-5 LP, and VES@5 for all compared frameworks. Best and second-best results in each task and metric group are boldfaced and underlined, respectively.

Task	Mean LP					Best LP					VES@5				
	DS	AIDE	DI	AK	SACK	DS	AIDE	DI	AK	SACK	DS	AIDE	DI	AK	SACK
FSS	0.0873	0.1441	0.1262	0.1331	0.1275	0.1201	<u>0.1980</u>	0.1396	0.1455	0.1984	0.5556	1	0.5556	1	<u>0.8333</u>
BPC	0.5004	0.6947	0.7690	<u>0.7951</u>	0.8019	0.8282	<u>0.8311</u>	0.8255	0.8296	0.8901	0.625	1	0.5556	<u>0.7143</u>	1
BPRD	0.1759	0.2776	0.2689	<u>0.3244</u>	0.7651	0.2840	0.3506	0.2814	0.3561	0.9032	0.4167	<u>0.8333</u>	0.7143	1	1
PPLT	0.2585	0.5025	0.4134	<u>0.5669</u>	0.6559	0.3707	<u>0.8317</u>	0.6547	<u>0.7393</u>	0.8628	0.625	<u>0.8333</u>	0.5556	1	0.7143
PCE	0.1674	0.1678	0.1673	<u>0.1981</u>	0.3632	0.1925	0.2039	0.1722	<u>0.2078</u>	0.5343	0.625	<u>0.8333</u>	1	0.625	1
POF	0.3242	0.3838	0.2965	<u>0.3056</u>	<u>0.3536</u>	0.5155	<u>0.7017</u>	0.3433	<u>0.3682</u>	0.7134	<u>0.7143</u>	1	<u>0.7143</u>	0.625	1
PIE	0.3652	0.5156	0.4316	<u>0.6214</u>	0.6453	0.6343	<u>0.7953</u>	0.6343	0.8720	0.8720	<u>0.8333</u>	0.7143	0.4545	<u>0.8333</u>	1
BCBD	0.3295	0.3241	0.3180	<u>0.3396</u>	0.3557	0.3703	<u>0.3899</u>	0.3364	0.3697	0.4722	0.7143	1	0.625	1	<u>0.8333</u>
PBMS	0.1322	0.0309	0.0611	0.2495	<u>0.2201</u>	0.1755	0.0356	0.0674	<u>0.3212</u>	0.3847	0.4167	<u>0.7143</u>	0.625	1	0.625
PRAR	0.2800	0.3451	0.3117	<u>0.3519</u>	0.4532	0.3248	<u>0.4204</u>	0.3381	<u>0.4204</u>	0.6379	0.625	<u>0.7143</u>	0.5556	0.4545	1
GGGB	0.0826	0.2760	0.1012	<u>0.3339</u>	0.5072	0.2031	0.9620	0.1874	0.6173	<u>0.9279</u>	0.5	0.8333	<u>0.7143</u>	0.8333	0.8333
TBOP	0.4486	0.5693	0.5213	<u>0.5773</u>	0.6539	0.7168	0.9061	0.6882	0.7305	<u>0.8287</u>	0.2778	<u>0.7143</u>	0.8333	<u>0.7143</u>	0.625
NYCTFP	0.3392	Fail	<u>0.3852</u>	<u>0.3833</u>	0.5663	<u>0.6918</u>	Fail	0.5347	0.4801	0.8941	<u>0.4545</u>	0	0.3846	<u>0.4545</u>	0.5
BSD	0.5624	0.6904	0.6777	<u>0.7499</u>	0.8375	0.8705	<u>0.9118</u>	0.7551	0.8544	0.9522	0.5556	<u>0.8333</u>	1	<u>0.8333</u>	0.7143
CD	0.2656	0.3191	0.2906	<u>0.3290</u>	0.3819	0.3513	<u>0.4253</u>	0.3296	0.4245	0.5239	0.625	<u>0.8333</u>	0.625	0.5	1
VPP	0.2504	0.2558	0.2391	<u>0.2626</u>	0.3031	0.3413	<u>0.3493</u>	0.2630	0.3036	0.4261	0.4167	1	0.5556	0.2778	<u>0.625</u>
CMIDSS	Fail	Fail	<u>0.4005</u>	0.3938	0.6365	Fail	Fail	0.4395	0.4864	0.9291	0	0	0.5556	0.4167	<u>0.4545</u>
PSSDP	0.2642	0.2300	0.2431	<u>0.3533</u>	0.5645	0.3487	0.2909	0.2605	<u>0.5283</u>	0.7688	0.4167	0.625	<u>0.7143</u>	0.5	0.8333
SCTP	0.4713	<u>0.3893</u>	0.3712	0.2231	0.2424	0.9675	<u>0.5531</u>	0.4810	0.2253	0.3066	0.7143	<u>0.625</u>	0.4167	0.7143	0.7143
ST	0.7265	0.2738	0.6840	0.5192	0.4825	<u>0.9410</u>	0.5350	0.9655	0.6898	0.7943	<u>0.8333</u>	1	0.625	<u>0.8333</u>	1

Note. Fail indicates that the corresponding framework failed to obtain the required five valid submissions within the retry budget; hence task-level Mean LP and Best LP are undefined rather than zero-valued leaderboard results. VES@5 is 0 by definition when the required valid submissions are not obtained within the retry budget. For aggregate comparisons, Fail entries are assigned 0 before macro-averaging LP-based metrics, yielding failure-penalized aggregate results.

directly observable from the raw data state, such as missingness patterns, data quality issues, basic distributional behavior, and coarse dimensionality characteristics. In-depth EDAInsight captures higher-order analytical signals that become more meaningful after basic cleaning, such as feature relationships, interaction structure, complexity-related patterns, and special scenarios. The same extraction pipeline is applied to historical cases during off-line construction and to the current task during online execution, so historical and current EDA signals are represented in a shared schema rather than in inconsistent ad hoc reports.

Table S6 lists representative EDAInsight fields rather than the full implementation vocabulary. These examples convey the analytical scope of the schema and its role in retrieval.

These structured EDA signals are later materialized into the case knowledge graph as competition-linked objects, allowing them to be retrieved jointly with task profiles, structural profiles, pipelines, and CoreInsights.

B.3 CoreInsight Extraction and Grounding

For each historical pipeline, CoreInsight extraction follows a three-stage offline procedure. First, statement-level nodes in the abstracted

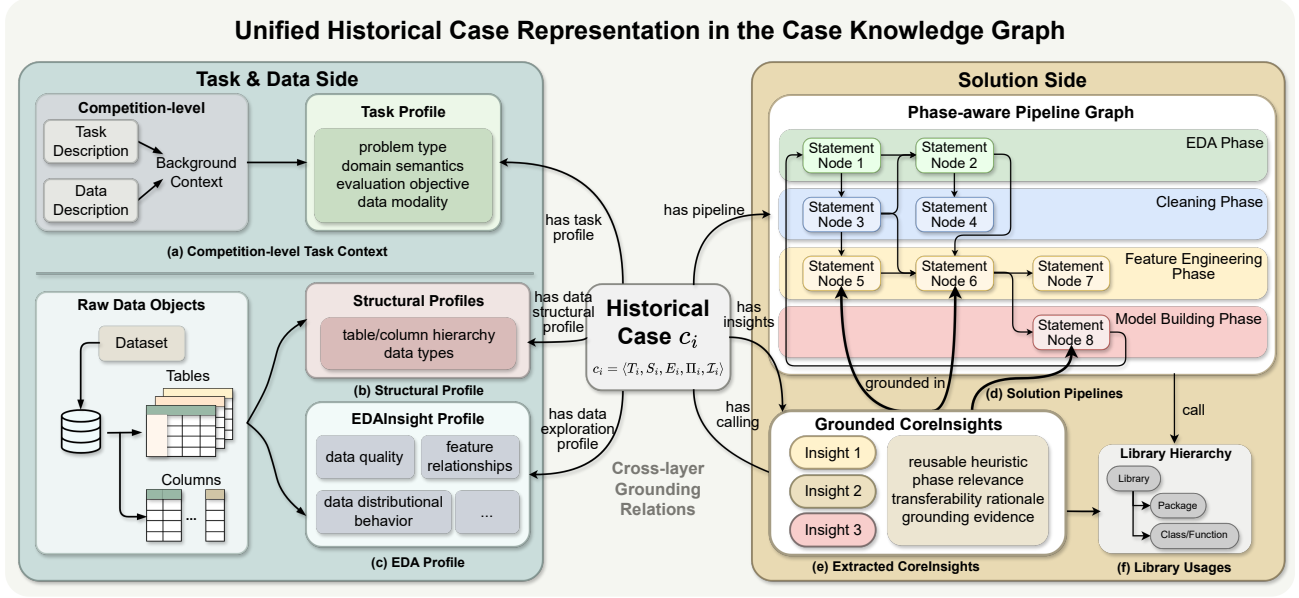


Figure S1: Case representation in SACK. Each historical case links task context, structural data profiles, EDAInsight profiles, phase-aware pipeline statements, library usages, grounded CoreInsights, and implementation evidence in a single case-centric knowledge substrate.

Table S5: Main object types in the SACK case knowledge graph.

Object type	Stores	Used for
Competition node	task context, overview, data description, structured elements	case-level retrieval
Table / Column node	structural organization, column metadata, representations	structural similarity and data-side comparison
Preliminary EDAInsight	raw-data quality and basic distribution signals	early-phase retrieval
In-depth EDAInsight	cleaned-data relationships and complexity signals	later-phase retrieval
Pipeline node	historical solution identity and metadata	solution-side traversal
Statement node	implementation-level code structure and phase labels	code grounding
CoreInsight node	reusable historical insight and rationale	planner-side guidance and developer-side grounding

pipeline graph are aligned with workflow phases such as EDA, Data Cleaning, Feature Engineering, and Model Building. Second, the pipeline is summarized at the phase level, yielding a compact representation of phase purposes, node ranges, major activities, and overall phase flow. Third, reusable historical insights are extracted from this phase-level summary together with their implementing node ranges and effectiveness rationales. This procedure turns raw

historical code into phase-aware and transferable knowledge units without losing traceability to the original implementation.

Formally, for a historical pipeline Π_i , the extraction procedure produces a set of grounded insights

$$I_i = \{I_{i1}, I_{i2}, \dots, I_{im}\}, \quad (S3)$$

where each insight is associated with one phase or multiple spanned phases and linked to a subset of statement nodes in Π_i that implement the corresponding historical pattern.

We use four insight types. DS denotes general data-science practice, DOM denotes domain- or task-specific reasoning, OPTIM denotes efficiency or performance optimization, and ARCH denotes architecture or workflow-organization patterns. Each extracted CoreInsight is stored as a graph node and linked to its originating case, pipeline, phase, and implementing statements. A defining property of CoreInsight is that it is not an ungrounded summary. Each CoreInsight is explicitly linked to the statement nodes that implement it in the historical pipeline. This grounding relation supports planner-side historical referencing and developer-side code grounding. Most CoreInsights are phase-local, but SACK also allows cross-phase insights when a reusable solution pattern spans multiple workflow stages. The purpose of offline extraction is not to reconstruct every low-level intent of the original solution author, but to identify transfer-relevant historical patterns with explicit implementation grounding.

B.4 CoreInsight Admissibility Checks

Before an extracted CoreInsight is stored in the case catalog, SACK applies admissibility checks to reduce ungrounded, phase-inconsistent, or redundant historical advice. These checks are not intended to

Table S6: Representative EDAInsight modules and fields.

Level	Module	Representative Fields
Preliminary	Data quality	missingness, outlier ratio, row completeness, integrity issues
Preliminary	Basic distribution	skewness, scale characteristics, multimodality, categorical imbalance
Preliminary	Basic dimensionality	number of samples, number of features, samples-per-feature ratio
In-depth	Feature relationships	correlation structure, multicollinearity, target relationships, interactions
In-depth	Complexity	sparsity, noise level, high dimensionality, feature redundancy
In-depth	Special scenarios	temporal patterns, group structure, spatial properties, high-cardinality effects

prove that every retained insight is optimal for future tasks. Instead, they enforce minimum requirements for traceability and transfer relevance: an insight must be tied to a workflow phase, grounded in implementation evidence, connected to observable task or data conditions, and not be a near-duplicate of an already stored insight. Table S7 summarizes the checks used during offline case construction.

C Stage-Adaptive Retrieval Details

This appendix provides additional details on SACK retrieval. The main paper presents stage-adaptive retrieval as a recall-rerank pipeline that transforms broad historical similarity into a filtered set of phase-appropriate CoreInsights. Here we clarify the signals used in shallow recall, the role of EDA-aware reranking, and the logic of Hybrid Applicability Gating.

For structural similarity, SACK performs table-column alignment over type-compatible columns. Column-level comparisons combine name-oriented and content-oriented representations, and table-level similarity aggregates the best matched columns. The resulting structural score is used as one component of the case-level recall score described in the main paper.

C.1 Retrieval Algorithm

Algorithm S1 summarizes the online stage-adaptive retrieval procedure used by SACK.

C.2 Shallow Similarity and Structural Matching

Shallow similarity is used for broad case-level recall rather than final historical reuse decisions. Its role is to screen a candidate set of historically plausible cases by combining task-side and data-side evidence. In SACK, this broad screening integrates three complementary signals: task-context similarity, coarse data-description similarity, and structural similarity over tables and columns.

Task-context similarity compares the current and historical competitions using competition descriptions and schema-constrained task attributes. This signal preserves broad semantic relevance at the case level. Coarse data-description similarity provides a lightweight comparison of how the data are described, helping distinguish cases that share task wording but differ in tabular setting. Structural similarity compares current and historical cases at table and column granularity. At a high level, this comparison is type-aware and uses both name-oriented and content-oriented column representations; table-level similarity is obtained by aggregating column-level matches, and competition-level structural similarity is derived from the resulting table alignments.

Algorithm S1 Stage-Adaptive Retrieval in SACK

Require: Current task state $x = \langle T_x, S_x, E_x, p \rangle$, historical case set C , recall size M , output size K

Ensure: Filtered CoreInsight pool $\hat{I}_p$

1: Compute shallow similarity $s_{\text{shallow}}(x, c_i)$ for each historical case $c_i \in C$

2: Recall candidate cases:

$$C_p^{\text{rec}} = \text{TopM}_{c_i \in C} s_{\text{shallow}}(x, c_i)$$

3: **for all** $c_i \in C_p^{\text{rec}}$ **do**

4: Compute phase-aware EDA similarity $s_{\text{EDA}}^{(p)}(x, c_i)$

5: **end for**

6: Rerank recalled cases:

$$C_p^{\text{top}} = \text{TopK}_{c_i \in C_p^{\text{rec}}} s_{\text{EDA}}^{(p)}(x, c_i)$$

7: Initialize candidate insight pool $I_p^{\text{cand}} \leftarrow \emptyset$

8: **for all** $c_i \in C_p^{\text{top}}$ **do**

9: Select strong historical pipelines $\text{TopPipelines}(c_i)$

10: **for all** $\pi \in \text{TopPipelines}(c_i)$ **do**

11: Retrieve Insights(π) and add them to I_p^{cand}

12: **end for**

13: **end for**

14: Apply phase filtering:

$$I_p^{\text{phase}} = G_{\text{phase}}^{(p)}(I_p^{\text{cand}})$$

15: Apply hard applicability gate:

$$I_p^{\text{hard}} = G_{\text{hard}}^{(p)}(I_p^{\text{phase}})$$

16: Apply soft applicability gate:

$$\hat{I}_p = G_{\text{soft}}^{(p)}(I_p^{\text{hard}})$$

17: **return** $\hat{I}_p$

Shallow similarity is intentionally recall-oriented. It is not designed to make a precise transfer judgment by itself. Instead, it produces a candidate set of plausible historical cases before later analytical refinement.

C.3 Phase-aware EDA Reranking

After broad case recall, SACK applies a second selection step based on EDAInsight. Unlike shallow similarity, which operates mainly on task semantics and coarse tabular structure, EDA-aware reranking focuses on whether the data behavior of a historical case is compatible with the current workflow phase.

This comparison is hierarchical rather than flat. Individual analytical fields are compared using representation-aware similarity

Table S7: Admissibility checks used before storing extracted CoreInsights in the case catalog.

Check	Purpose	Rejection or revision condition
Phase consistency	Preserve workflow-stage meaning	Insight phase conflicts with the phase labels of its grounding statements
Evidence grounding	Avoid ungrounded LLM-style advice	Insight lacks originating case, pipeline fragment, implementing statements, or relevant data-flow/library context
Contextual applicability	Retain transfer-relevant evidence	Rationale does not refer to observable task, schema, EDA, validation, or metric conditions
Redundancy control	Reduce repeated generic insights	Near-duplicate insights share the same phase, rationale, and statement grounding

functions appropriate to their types, such as scalar statistical indicators, boolean-style signals, or structured distributional fields. These field-level comparisons are aggregated into module-level similarities, and the visible modules are combined into a phase-aware EDA similarity score. This design reflects that analytical compatibility in tabular workflows is often expressed through structured combinations of local signals rather than a single global representation.

A key design principle is *phase visibility*. Earlier workflow stages mainly rely on signals observable from raw or lightly processed data, especially those captured by `pre_eda`, such as data quality and basic distribution. Later stages can additionally use higher-order signals from `deep_eda`, including feature relationships, interaction structure, complexity, and special scenarios. SACK therefore computes EDA-aware reranking using only the modules visible at the current phase. Because the same EDAInsight schema is used for historical cases and the current task, reranking is performed in a shared analytical space rather than through free-form textual comparison over EDA summaries.

EDA-aware similarity does not replace broad recall. It refines the recalled candidates so that cases whose analytical behavior better matches the current phase are ranked higher before candidate CoreInsight collection. Table S8 summarizes the EDAInsight modules visible to retrieval and gating at each phase. In our implementation, phase-specific retrieval weights determine how much shallow case similarity and EDAInsight similarity contribute to ranking. Early phases rely more on task and structural similarity, whereas later phases increasingly use EDAInsight signals that describe data behavior and feature relationships. This phase-dependent composition implements the stage-adaptive retrieval described in the main paper.

C.4 Hybrid Applicability Gating

The Hybrid Gate is a conservative filtering mechanism. It does not guarantee that every retained insight is optimal; rather, it removes candidates that are phase-incompatible, rule-incompatible, or semantically weak for the current task.

The soft gate uses a structured output interface:

```
{
  "candidate_id": "...",
  "decision": "keep | warn | drop",
  "confidence": 0.0,
  "reason": "...",
  "risk_tags": ["..."],
```

Table S8: Phase-visible EDAInsight modules.

Workflow Phase	Visible EDAInsight	Retrieval Emphasis
Preliminary EDA	Task and structural profiles; optional preliminary signals	Broad case recall and exploratory guidance
Data Cleaning	Preliminary EDAInsight	Missingness, outliers, integrity, basic distributions
In-depth EDA	Preliminary EDAInsight and selected in-depth modules	Data behavior and feature relationships
Feature Engineering	Preliminary and in-depth EDAInsight	Feature relationships, interactions, sparsity, special scenarios
Model Building	Preliminary and in-depth EDAInsight	Validation design, modeling constraints, task-specific patterns

```
"adaptation_hint": "..."}
}
```

Candidates marked as `keep` are exposed directly to the planner context. Candidates marked as `warn` are retained but accompanied by an adaptation hint. Candidates marked as `drop` are removed before downstream evidence exposure.

C.5 Retrieval Hyperparameters and Online Use Statistics

Table S10 complements the offline case-base statistics by summarizing how historical case knowledge is used during online execution.

For each task t , run r , and phase p , SACK produces a representation–retrieval–evidence exposure chain:

$$\mathcal{I}_{t,r,p}^{cand} \rightarrow \mathcal{I}_{t,r,p}^{phase} \rightarrow \mathcal{I}_{t,r,p}^{hard} \rightarrow \mathcal{I}_{t,r,p}^{soft} \rightarrow \mathcal{I}_{t,r,p}^{plan} \rightarrow \mathcal{I}_{t,r,p}^{code}.$$

Here, $\mathcal{I}^{cand}$ denotes candidate CoreInsights collected from retrieved cases, $\mathcal{I}^{phase}$ denotes phase-aligned insights, $\mathcal{I}^{hard}$ and $\mathcal{I}^{soft}$ denote insights surviving hard and soft applicability gates, $\mathcal{I}^{plan}$ denotes CoreInsights explicitly referenced by the planner, and $\mathcal{I}^{code}$

Table S9: Hybrid Gate component examples.

Gate	Removes	Example
Phase gate	Insights from mismatched workflow phases	Modeling insight during preliminary EDA
Hard gate	Operations forbidden in the current phase	Data modification during an EDA-only phase
Hard gate	Validation strategies conflicting with task constraints	Random split for time-series tasks
Hard gate	Strategies conflicting with grouping requirements	Ordinary K-fold when group-aware validation is required
Soft gate	Semantically weak or unsupported insights	Insight not supported by current EDA signals
Soft gate	Insights requiring nontrivial adaptation	Retained with warning and adaptation hint

Table S10: Online retrieval and knowledge reuse statistics of SACK. Values are averaged over valid SACK runs.

Statistic	Value
Recalled historical cases per phase	20.00
Reranked top cases per phase	5.00
Selected historical pipelines per phase	11.84
Candidate CoreInsights per phase	33.72
Phase-aligned CoreInsights per phase	26.91
Hard-gate-retained CoreInsights per phase	19.63
Soft-gate kept CoreInsights per phase	6.82
Soft-gate warning CoreInsights per phase	5.07
Soft-gate dropped CoreInsights per phase	7.74
Planner-adopted CoreInsights per phase	6.41
Developer-grounded CoreInsights per phase	5.72
Candidate-to-retained ratio	35.26%
Planner adoption rate	53.91%
Developer grounding coverage	89.24%
Soft-gate drop ratio	39.43%

Note. Candidate CoreInsights are collected after case-level recall and phase-aware reranking. Retained CoreInsights include those kept or warned by the soft gate after phase filtering and hard gating. Adopted CoreInsights are explicitly referenced in planner-generated phase plans, and grounded CoreInsights are adopted references resolved to implementation evidence for developer-side code generation.

denotes adopted insights resolved to implementation evidence for developer-side generation.

We report three utilization ratios and one filtering diagnostic ratio:

$$R_{\text{ret}} = \frac{|I^{\text{soft}}|}{|I^{\text{cand}}|}, \quad R_{\text{adopt}} = \frac{|I^{\text{plan}}|}{|I^{\text{soft}}|}, \quad R_{\text{ground}} = \frac{|I^{\text{code}}|}{|I^{\text{plan}}|}.$$

These ratios measure the retained knowledge fraction, planner adoption rate, and developer grounding coverage, respectively.

D Role-Aware Evidence Exposure Details

D.1 Evidence Exposure Algorithm

Algorithm S2 summarizes how retained CoreInsights are exposed differently to the planner and developer roles.

D.2 Planner-side Evidence Format

Planner-side evidence exposure and developer-side implementation grounding use different interfaces.

For the planner, each retained CoreInsight is formatted as strategy-level context:

Algorithm S2 Role-Aware Evidence Exposure in SACK

Require: Current task state x , current phase p , filtered CoreInsight pool $\hat{I}_p$, historical phase memory $H_{<p}$

Ensure: Phase plan $Plan_p$, phase code $Code_p$

1: Transform filtered insights into planner-oriented heuristic context:

$$Z_p^{\text{plan}} = \Phi_{\text{plan}}(\hat{I}_p)$$

2: Generate phase plan:

$$Plan_p = \Psi_{\text{plan}}(x, p, H_{<p}, Z_p^{\text{plan}})$$

3: Parse explicit historical references from the generated plan:

$$Ref_p = \{(c_i, \pi_j, I_{jk}) \mid I_{jk} \text{ in } Plan_p\}$$

4: Retrieve developer-side grounded evidence:

$$Z_p^{\text{code}} = \Phi_{\text{code}}(Ref_p)$$

5: Generate phase code:

$$Code_p = \Psi_{\text{dev}}(Plan_p, x, p, Z_p^{\text{code}})$$

6: **return** $Plan_p, Code_p$

Similar Competition: <competition_id>

Pipeline: <pipeline_id>

Insight: <insight_id>

Phase: <phase>

Description: <insight_description>

Rationale: <effectiveness_rationale>

Gate Status: keep | warn

Adaptation Hint: <optional_hint>

If the planner adopts an insight, it records an explicit reference in the phase plan:

Referenced from:

Competition[<competition_id>]

-> Pipeline[<pipeline_id>]

-> Insight[<insight_id>]

D.3 Developer-side Grounding Format

For the developer, SACK resolves each reference back to the case-knowledge graph and retrieves the implementing statements linked to the corresponding CoreInsight:

Task: <planner_step>

Referenced Insight: <competition_id>/<pipeline_id>/<insight_id>

Grounded Code Evidence:

[1] <statement_text>

[2] <statement_text>

Table S11: Runtime environment and model endpoints used in the implementation.

Component	Role in SACK
Local machine	Intel i7-12700 CPU and 32GB memory; used for workflow execution, code running, model training, validation, and submission generation.
qwen3-max	Used for complex reasoning tasks, including task understanding, planning, evaluation interpretation, and report generation.
qwen3-coder-plus	Used for code generation, debugging, and iterative code refinement.
qwen3-vl-flash	Used when visual or chart-based interpretation is required.
text-embedding-v3	Used to build semantic representations for retrieval and embedding-based matching.
GraphDB	Used to store and query the RDF-style case knowledge graph.
PostgreSQL / pgvector	Used for vector indexing and embedding-based retrieval.

...

This interface conditions developer-side code evidence on planner-side historical adoption, rather than retrieving independently from all historical code artifacts.

E Runtime and Implementation Details

E.1 Runtime Environment and Model Endpoints

The experiments use a hybrid local-and-cloud environment. The local machine is equipped with an Intel i7-12700 CPU and 32GB memory, and is responsible for project execution, code running, downstream model training, validation, and submission generation. LLM-based components are accessed through cloud API services. Different model endpoints are used for different system roles, as summarized in Table S11.

This setup separates SACK-specific retrieval and evidence exposure from downstream task execution. The offline case catalog is constructed once before evaluation, while online task execution repeatedly queries the fixed case catalog during workflow phases. The runtime costs of these two parts are reported separately in Appendix E.2 and Appendix E.3.

E.2 Offline Case-catalog Construction Cost

SACK separates offline case-catalog construction from online task execution. Table S12 reports the offline case-catalog construction statistics and timing under the default lightweight inference setting. The offline stage is performed once before evaluation and amortized across downstream tasks.

E.3 Online Retrieval Cost

During online execution, SACK separates case-level retrieval from full stage retrieval. Table S13 reports the online retrieval cost breakdown. Case-level retrieval can be reduced to sub-second latency

Table S12: Case-catalog construction statistics and timing under the default lightweight inference setting.

Item	Value
Historical case directories	79
Cases with CSV files	78
Cases with complete pre/deep EDAInsight	70
Notebook-backed historical cases	69
Source notebook pipelines	695
Pipeline metadata files	669
CoreInsight analysis files	617
Column/profile JSON files	12,510
Pipeline TTL total size	278.41 MB
Average pipeline-abstraction time	37.46 s / pipeline
Total Pipeline-abstraction time for pipelines	6.96 h
Single-case profiling time	142.11 s
Estimated profiling time for 69 cases	2.72 h
Global schema construction time	15.13 s
GraphDB schema import time	12.99 s
GraphDB pipeline upload time	164.24 s
PostgreSQL embedding DB creation time	1.24 s
Embedding population time	55.43 s
Estimated total offline construction time	9.8 h

with cached current-task profiles and similarity results, while full stage retrieval is mainly affected by CoreInsight fetching and the LLM-assisted soft gate.

F Additional Task-level Analyses

F.1 Task-level Ablation Patterns

Figure S2 reports task-level ablation patterns across benchmark tasks. The figure complements the aggregate ablation results in the main paper by showing how removing case catalog reuse, stage-adaptive retrieval, or role-aware evidence exposure affects leaderboard competitiveness across individual tasks.

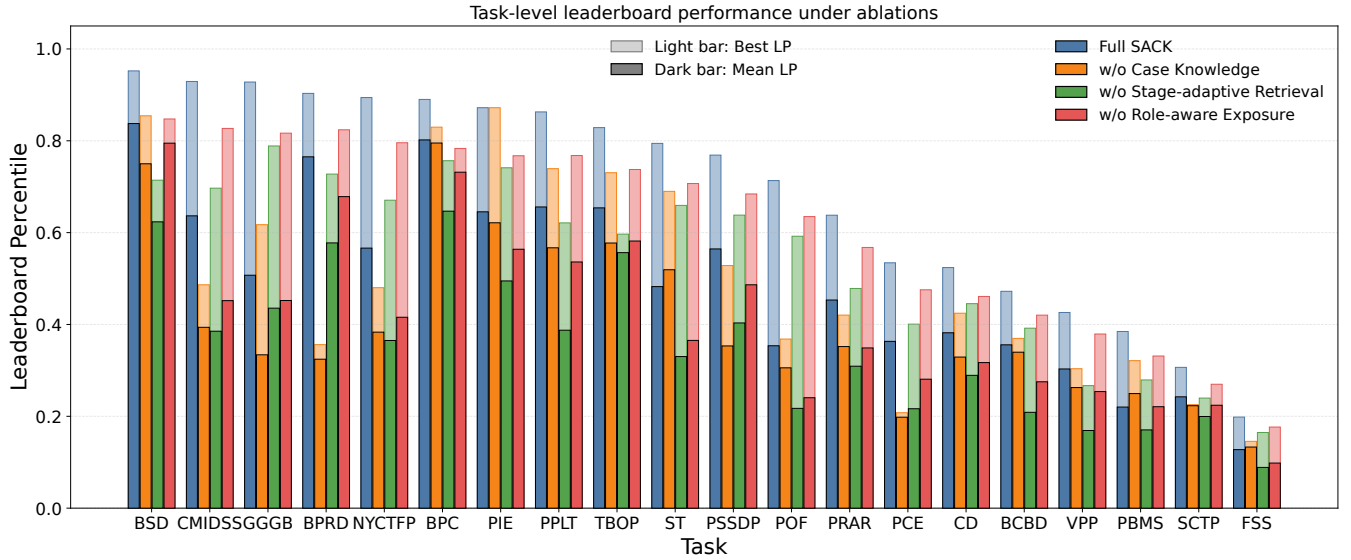
F.2 Supporting Evidence for Representative Task Analyses

This appendix provides supporting evidence for the representative task analyses discussed in the main paper. The three representative tasks are not expanded at the same level of detail because they serve different diagnostic purposes. BPRD is used to illustrate positive transfer, so we provide a compact phase-level trace summary of adopted historical guidance. SCTP is used both as a limited-gain case and as a concrete example of role-aware traceability, so we provide the leaderboard-neighborhood summary and a more detailed reuse trace. PBMS is used as a negative-transfer case, so we summarize the risk signals observed in the generated phase plans rather than reconstructing the full trace.

F.2.1 BPRD: Positive-transfer Trace Summary. BPRD is used as a positive-transfer case in the main paper. The goal of this subsection is not to reconstruct the full planning log, but to summarize the phase-level evidence showing that the gain comes from a coherent sequence of historically grounded decisions rather than a

Table S13: Online retrieval cost breakdown in SACK. Case-level retrieval denotes similar-case recall and phase-aware reranking. Full stage retrieval additionally includes CoreInsight fetching, applicability filtering, and the LLM-assisted soft gate.

Online component	Cold retrieval	Cached stage reuse
Current-task profile generation	Seconds to tens of seconds	Skipped or reused
Base/EDA similarity query	Tens of seconds	Skipped or reused
Load cached similar-case table	–	< 0.5 s
Phase-specific reranking	–	< 1 s
Top-case output	< 1 s	< 1 s
Case-level retrieval subtotal	≈ 30s	≈ 1s
GraphDB and pgvector queries	≈ 10s	5–10 s
CoreInsight fetching and organization	≈ 10s	5–10 s
LLM-assisted soft gate	≈ 30s	≈ 30s
Evidence-resolution and gating subtotal	≈ 50s	≈ 40s
Full first cold phase total	≈ 80s	–
Task-level SACK overhead	4–5 min	–

**Figure S2: Task-level ablation results across benchmark tasks. Light bars report Best-of-5 LP and dark bars report Mean-of-5 LP.**

single modeling trick. Table S15 summarizes the adopted guidance observed in the phase traces.

This trace summary supports the interpretation that BPRD benefits from cross-phase continuity. The retrieved evidence first guides cleaning decisions that respect meteorological semantics, then supports weather-specific feature construction, and finally remains active during validation and probability-oriented prediction. Thus, the positive transfer is caused by accumulated stage-wise decisions rather than by direct reuse of a historical notebook or a single code fragment.

F.2.2 SCTP: Leaderboard Compression and Complete Trace Example. SCTP is used as a limited-gain case and as a complete traceability example. The main paper reports that SACK does not outperform the strongest baseline on this task, but the phase traces show that historical evidence is still used in a structured and role-aware manner.

We therefore provide two complementary forms of evidence. Table S16 summarizes the leaderboard neighborhood, explaining why score-space differences and leaderboard-percentile differences are not linearly aligned. The subsequent trace illustrates how planner-side historical references are generated and how adopted insights can be resolved to developer-side grounding evidence.

SCTP is a binary classification task over anonymized tabular features, with AUC as the evaluation metric. Before each workflow phase, SACK retrieves and filters historical CoreInsights. The planner receives retained insights as strategy-level evidence and records explicit references to the originating case, pipeline, and insight when an insight is adopted. For readability and anonymization, we replace concrete historical competition names and pipeline identifiers with stable local identifiers. For example, Case[C-DC-01] → Pipeline[P-DC-01] → Insight[I-DC-01] denotes a retrieved historical case, one of its solution pipelines, and a CoreInsight

Table S14: Scope of supplementary evidence for representative task analyses.

Task	Role in analysis	Supplementary evidence provided
BPRD	Positive-transfer case	Compact phase-level trace summary showing how adopted historical guidance supports cleaning, feature engineering, and modeling.
SCTP	Limited-gain case and traceability example	Leaderboard-neighborhood summary plus a detailed role-aware trace showing planner-side adoption and developer-side grounding.
PBMS	Negative-transfer case	Risk-oriented evidence table summarizing historically guided decisions that may weaken task alignment.

adopted by the planner during the data-cleaning phase. The anonymization preserves the reference structure while removing dataset- and author-specific identifiers.

During data cleaning, the planner adopts retrieved CoreInsights for identifier removal, low-variance pruning, outlier treatment, train-test consistency, and post-cleaning validation. The first cleaning step removes the identifier column to avoid spurious correlations:

Remove ID column to prevent data leakage and model overfitting.
Referenced from: Case[C-DC-01] -> Pipeline[P-DC-01]
-> Insight[I-DC-01]

The retrieved insight is adapted to the current task by treating the identifier as a non-informative field and requiring identical processing for both training and test data. The planner also adopts a systematic outlier-handling insight:

Detect and handle outliers in training data using IQR-style clipping.
Referenced from: Case[C-DC-02] -> Pipeline[P-DC-02]
-> Insight[I-DC-02]

Instead of removing rows, the plan adapts the historical insight by using clipping, preserving sample size under class imbalance. A subsequent step applies the same training-derived thresholds to the test data:

Apply consistent outlier treatment to test data using training-derived thresholds.
Referenced from: Case[C-DC-03] -> Pipeline[P-DC-03]
-> Insight[I-DC-03]

This explicitly prevents leakage by forbidding threshold recomputation on the test set. Finally, the cleaning plan includes a lightweight post-cleaning validation step:

Generate validation visualizations to verify cleaning impact.
Referenced from: Case[C-DC-04] -> Pipeline[P-DC-04]
-> Insight[I-DC-04]

The insight is adapted by limiting diagnostics to representative features with meaningful cleaning effects, keeping the step efficient while checking that cleaning does not distort feature distributions.

The feature-engineering phase shows how retrieved CoreInsights are adapted to current EDA findings rather than copied as

fixed templates. For train-test divergence, the planner adopts a historical insight about distribution alignment:

Apply robust scaling and feature clipping to reduce train-test distribution divergence for highly divergent features.
Referenced from: Case[C-FE-01] -> Pipeline[P-FE-01]
-> Insight[I-FE-01]

The plan adapts this insight by focusing only on features whose divergence was identified in EDA and by requiring all thresholds and transformation parameters to be derived from the training data. For nonlinear target relationships, the planner references a historical insight about data-driven transformation:

Create optimized representations of features exhibiting strong nonlinear relationships with the target variable.
Referenced from: Case[C-FE-02] -> Pipeline[P-FE-02]
-> Insight[I-FE-02]

The historical insight is adapted into binned or piecewise feature representations for selected variables, with constraints on the number of segments to reduce overfitting. The planner also adopts targeted distribution transformation:

Apply targeted transformations to features with identified distribution pathologies.
Referenced from: Case[C-FE-03] -> Pipeline[P-FE-03]
-> Insight[I-FE-03]

Rather than applying transformations globally, the plan targets features whose pathologies are observed in EDA, such as skewness or boundary-heavy behavior. For feature selection, the planner uses a CoreInsight about correlation-aware filtering:

Select relevant features through two-stage correlation-aware feature selection.
Referenced from: Case[C-FE-04] -> Pipeline[P-FE-04]
-> Insight[I-FE-04]

The adaptation uses rank-based relevance checks for nonlinear relationships and adds pairwise correlation filtering to reduce redundancy. The phase ends with a strict feature-space consistency check:

Prepare final processed datasets with consistent feature spaces for model building.
Referenced from: Case[C-FE-05] -> Pipeline[P-FE-05]
-> Insight[I-FE-05]

This step requires identical feature spaces between training and test data before model building, supporting downstream execution stability.

The model-building phase shows that CoreInsight reuse is not limited to preprocessing. The planner also uses historical knowledge for imbalance diagnosis, validation design, model diversity, hyperparameter optimization, ensemble construction, and submission generation. The first modeling step analyzes class imbalance before applying correction techniques:

Analyze target distribution and assess class imbalance severity.
Referenced from: Case[C-MB-01] -> Pipeline[P-MB-01]
-> Insight[I-MB-01]; Case[C-MB-02] -> Pipeline[P-MB-02]
-> Insight[I-MB-02]

Table S15: Compact phase-level trace summary for BPRD. The table reports the role of adopted historical guidance rather than a complete planning log.

Workflow phase	Adopted historical guidance	Role in positive transfer
Data cleaning	Domain-consistent handling of meteorological variables, train–test structural alignment, data-type harmonization, and plausibility-aware outlier treatment.	Prevents generic preprocessing from distorting weather-specific variables and keeps downstream feature construction consistent across train and test data.
Feature engineering	Construction of weather-aware features such as temperature-range features, humidity-related features, wind-direction encodings, and rainfall-condition interactions.	Converts raw meteorological columns into task-relevant predictors and adapts historical feature-engineering patterns to the current EDA findings.
Validation and modeling	Competition-aligned probability prediction, feature-space consistency checks, and validation choices aligned with the binary rainfall-prediction objective.	Preserves earlier cleaning and feature-engineering decisions through model building and final submission generation, rather than allowing later stages to drift into unconstrained model search.

Table S16: Leaderboard-neighborhood summary for SCTP. The table explains why the score-space disadvantage of SACK is translated into a less extreme leaderboard-space disadvantage.

Item	Value	Interpretation
Total teams	8751	Large competition with a dense and highly competitive leaderboard.
Winning score	0.9257	Indicates a strong top-end performance ceiling.
25% leaderboard score	0.8998	The upper leaderboard region is highly compressed.
Median leaderboard score	0.8978	A narrow score gap separates many teams around the middle-to-upper region.
75% leaderboard score	0.8558	The distribution begins to spread more noticeably below the upper region.
95% leaderboard score	0.5411	Tail quality drops sharply, showing a long lower-quality region.
Strongest baseline mean score	0.8921	Close to the dense upper-middle leaderboard band.
SACK mean score	0.8305	Weaker in score space, but not collapsed into the extreme tail.
Strongest baseline mean LP	0.4713	Competitive but still far from top leaderboard positions.
SACK mean LP	0.2424	Worse than the strongest baseline, but the leaderboard penalty is less extreme than the score-space gap alone would suggest.
Strongest baseline best score	0.9032	Approaches the dense upper leaderboard region.
SACK best score	0.8826	Remains competitive, but falls short of crossing into the stronger local score band.
Strongest baseline best LP	0.9675	Best-run competitiveness is high for the strongest baseline.
SACK best LP	0.3066	Best-run rank movement remains limited relative to the stronger baseline.

The plan requires a quantitative imbalance report before model training and applies imbalance handling only when the imbalance ratio exceeds a specified threshold. The next step constructs a leakage-aware validation framework:

```
Implement strategic data splitting and validation
framework setup.
Referenced from: Case[C-MB-02] -> Pipeline[P-MB-02]
-> Insight[I-MB-02]
```

The plan requires stratified train–holdout splitting and stratified cross-validation, and specifies that any sampling strategy must be applied within cross-validation folds rather than before splitting. For base model training, the planner uses historical insights about model diversity and evaluation:

```
Train diverse base models with appropriate imbalance
handling.
Referenced from: Case[C-MB-03] -> Pipeline[P-MB-03]
```

```
-> Insight[I-MB-03]; Case[C-MB-04] -> Pipeline[P-MB-04]
-> Insight[I-MB-04]
```

The retrieved insights are adapted into a resource-bounded set of diverse models, each with model-specific imbalance handling and AUC-based evaluation. The planner then performs targeted hyperparameter optimization:

```
Perform targeted hyperparameter optimization for
top-performing models.
Referenced from: Case[C-MB-01] -> Pipeline[P-MB-05]
-> Insight[I-MB-05]; Case[C-MB-05] -> Pipeline[P-MB-06]
-> Insight[I-MB-06]
```

This step limits optimization to the top-performing models and uses targeted search to preserve resources while optimizing the AUC objective. The final modeling steps construct and deploy an ensemble:

```
Build performance-weighted ensemble of top models
and generate final probability predictions.
```

```

Referenced from: Case[C-MB-07] -> Pipeline[P-MB-08]
-> Insight[I-MB-09]; Case[C-MB-08] -> Pipeline[P-MB-09]
-> Insight[I-MB-10]

```

The ensemble is adapted by weighting models according to validation AUC and using probability-range checks before submission generation.

This trace illustrates four properties of SACK. First, retrieved workflow evidence is used as transferable CoreInsights rather than copied as raw historical code. Second, each adopted insight is explicitly grounded by a reference chain of the form Case -> Pipeline -> Insight. Third, planner-side reuse is task-adaptive: the generated plan adds current-task constraints such as training-only threshold estimation, leakage-aware validation, resource-bounded tuning, and probability-range post-processing. Fourth, the trace spans multiple workflow phases, showing historical knowledge reuse for preprocessing, validation design, model selection, ensemble construction, and submission generation. The developer can then resolve these references to retrieve implementation evidence from the corresponding grounded statements.

This trace explains why SCTP is a limited-gain case rather than a no-transfer case. Historical evidence is adopted and grounded throughout the workflow, but its effect is primarily to stabilize robust tabular-processing decisions. On this highly competitive anonymized tabular task, such stabilization is not sufficient to exceed the strongest baseline.

F.2.3 PBMS: Negative-transfer Backing. PBMS is used as a representative unstable-transfer case. Unlike BPRD and SCTP, the goal of this subsection is not to reconstruct the full role-aware reuse trace. Instead, we summarize the historically guided decisions that may weaken alignment with the current task. Table S17 highlights the main risk signals observed in the generated phase plans.

Table S17: Negative-transfer evidence summary for PBMS.

Phase	Adopted decision	Risk in PBMS
Cleaning	Remove near-perfectly correlated features as leakage-like artifacts.	A highly predictive feature available in both train and test data is not necessarily leakage; removing it may discard useful signal.
Feature eng.	Apply music-theoretic clipping rules, tempo bins, and handcrafted interactions.	Synthetic data may only approximately match the music domain, so strong semantic priors may be over-imposed.
Feature eng.	Construct target-adjacent features such as tempo categories and BPM-normalized ratios.	Even if later removed, this indicates planning drift toward leakage-adjacent feature engineering.
Modeling	Use diversity-oriented subset design and correlation-aware ensembling.	This complexity may be unnecessary for the synthetic regression task and may distract from stronger direct baselines.

This evidence supports the main-paper interpretation that negative transfer can occur when historically plausible workflow patterns are only weakly supported by the current task state. In PBMS,

the retrieved evidence encourages an over-engineered path: it suppresses potentially useful correlated signals, imposes domain-specific transformations on synthetic tabular features, and increases modeling complexity. The case therefore illustrates the residual limitation of stage-adaptive gating: it reduces harmful transfer but does not fully eliminate assumptions that are plausible in historical cases yet weakly aligned with the current data.